Mathematics of Machine Learning

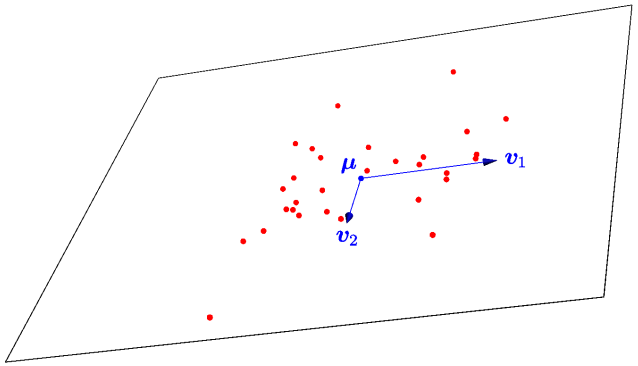
Matrix Factorisation

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	∅	4	5	2
Bob	5	3	∅	3	∅
Chas	∅	2	3	5	3

Matrix completion, low-rank approximation, recommender systems, alternating least squares, non-negative matrix factorisation

Outline

1. **Missing Data and Recommenders**
2. Low-Rank Approximation
3. Probabilistic Approach
4. Non-Negative Matrix Factorisation



Matrix Completion

- It is extremely common to be given a matrix of data with missing entries (particularly when the data comes from humans filling in forms)
- Most learning machines need a complete set of data, so we either throw away examples or features with missing values (sometimes leaving nothing) or do our best to infer them
- Inferring them only works if there is structure in the matrix that we can exploit
- The structure we use in this lecture: **structured data typically lies close to a low-dimensional subspace** (the idea behind PCA and the SVD)

Recommender Systems

- The best-known example: recommending items to users, which can be financially important
  - ⋆ Recommendations are reported to drive a large share of Amazon's sales
  - ⋆ Netflix (then a DVD rental company, now a streaming service) ran a $1 million competition (2006–09) to improve the quality of its recommendations
- **Content-based filtering** uses (often text) features to find similar items; **collaborative filtering** uses ratings supplied (implicitly or explicitly) by users
- We concentrate on collaborative filtering

Recommending Movies

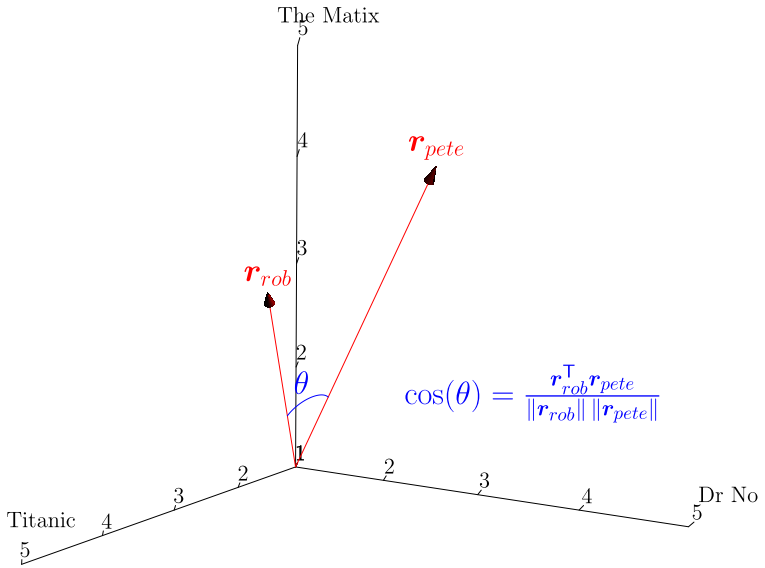
- The classic example is to recommend a movie given some ratings from users

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	Ø	4	5	2
Bob	5	3	Ø	3	Ø
Chas	Ø	2	3	5	3

- Ø means there are no ratings
- The task is to predict the unknown ratings and make a recommendation
- Typically data sets are huge and very sparse

Classic Collaborative Filtering

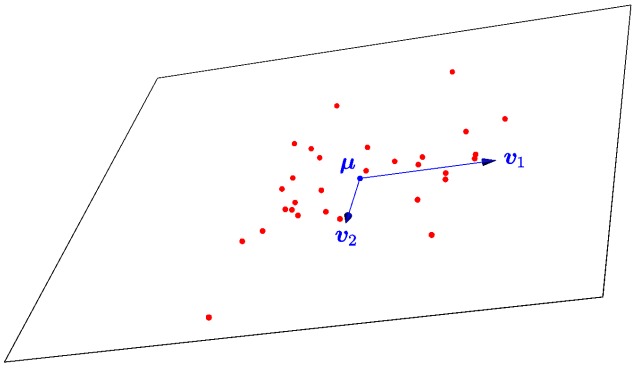
- The classic collaborative filtering technique is to find similar users (those giving similar ratings)
- Use the recommendation of similar users weighted by their similarity
- Can also recommend items that are similar (i.e. items that have received similar ratings) to those that a user rates highly



$$\hat{R}_{ui} = \frac{\sum_{u'} \mathrm{sim}(u, u') R_{u'i}}{\sum_{u'} \mathrm{sim}(u, u')}$$

Outline

1. Missing Data and Recommenders
2. **Low-Rank Approximation**
3. Probabilistic Approach
4. Non-Negative Matrix Factorisation



Matrix Completion

- A different (often better) method is to think of recommendation as a matrix completion task

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	Ø	4	5	2
Bob	5	3	Ø	3	Ø
Chas	Ø	2	3	5	3

- Complete the matrix

$$\mathbf{R} = \begin{pmatrix} 2 & & 4 & 5 & 2 \\ 5 & 3 & & 3 & \\ & 2 & 3 & 5 & 3 \end{pmatrix}$$

Replace by Means

- The simplest guess uses the means, column-wise and row-wise

$$R_{ui} \approx \bar{R}_u + \bar{R}_i - \bar{R}$$

  where

  - $\bar{R}_u$ is the mean rating of user u
  - $\bar{R}_i$ is the mean rating for item i
  - $\bar{R}$ is the mean overall rating

- Gets the means right and gives a reasonable rating
- (For general missing data: replace by the feature's mean)
- But this ignores structure: similar users give similar ratings

Best Low-Rank Approximation

- Recall the SVD of a complete matrix, with $s_1 \geq s_2 \geq \cdots \geq 0$

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \sum_k s_k \boldsymbol{u}_k \boldsymbol{v}_k^\mathsf{T}$$

- Keeping the K largest singular values gives a rank-K matrix $\mathbf{X}_K = \sum_{k=1}^{K} s_k \boldsymbol{u}_k \boldsymbol{v}_k^\mathsf{T}$
- **Eckart–Young theorem**: $\mathbf{X}_K$ is the best rank-K approximation to $\mathbf{X}$, with error $\|\mathbf{X} - \mathbf{X}_K\|_F^2 = \sum_{k>K} s_k^2$ (also best in the spectral norm, with error s_{K+1})
- Data close to a K-dimensional subspace has small discarded s_k (this is effectively PCA)

Completing a Matrix with the SVD

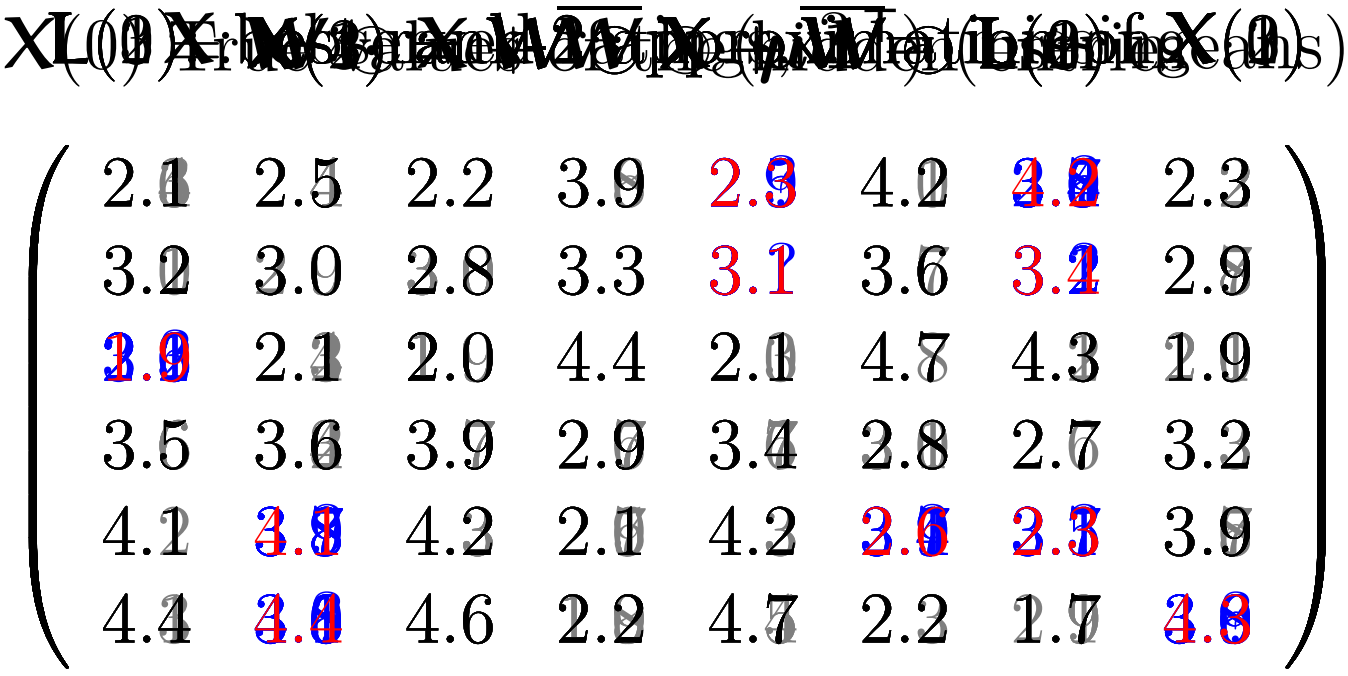
- The SVD needs a complete matrix, so we iterate
  1. Fill in the missing entries (e.g. with the user means)
  2. Compute the best rank-K approximation $\mathbf{L}$ of the filled-in matrix
  3. Replace the missing entries (only) by the values in $\mathbf{L}$
  4. Repeat from 2

- With a mask $\boldsymbol{W}$ ($W_{ij} = 1$ where X_{ij} is observed, 0 otherwise) and $\odot$ the element-by-element product

$$\mathbf{X}(t+1) = \boldsymbol{W} \odot \mathbf{X} + \overline{\boldsymbol{W}} \odot \mathbf{L}(t)$$

  where $\overline{\boldsymbol{W}} = 1 - \boldsymbol{W}$

SVD in Operation



Low-Rank Approximation

- More directly, approximate an $m \times n$ matrix $\mathbf{X}$ by

$$\mathbf{X} \approx \mathbf{A}\,\mathbf{B}$$

  where $\mathbf{A}$ is $m \times K$ and $\mathbf{B}$ is $K \times n$

- $\mathbf{AB}$ has $rank$ at most K

- We replace $m \times n$ elements by $(m+n) \times K$ free parameters

- K controls the complexity of the model: large K fits the data better but may over-fit

- Choose K by cross-validation (hide some of the observed entries and see how well we predict them)

Low-Rank Updates: LoRA

- **LoRA** (low-rank adaptation, Hu et al. 2021) uses the same idea to fine-tune a large network: a pretrained weight matrix $\boldsymbol{W}_0$ $(m \times n)$ becomes $\boldsymbol{W}_0 + \Delta\boldsymbol{W}$
- LoRA freezes $\boldsymbol{W}_0$ and learns only a low-rank change

$$\Delta\boldsymbol{W} = \mathbf{A}\mathbf{B}, \qquad \mathbf{A}\ (m \times K),\ \mathbf{B}\ (K \times n),\ K \ll m,n$$

  with one factor starting at zero (so training starts from $\boldsymbol{W}_0$)
- For $m = n = 4096$ and $K = 8$: $(m+n)K \approx 6.6 \times 10^4$ parameters to train instead of $mn \approx 1.7 \times 10^7$ (0.4%)
- The bet: the change a new task needs is close to low rank; each task stores only its own $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{AB}$ can be added into $\boldsymbol{W}_0$ afterwards

Least-Squares

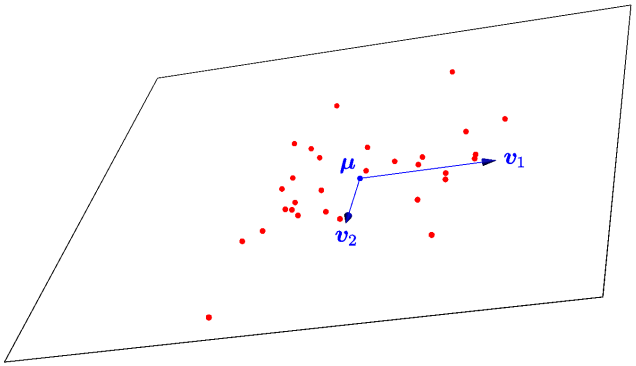
- Assume we have elements of $\mathbf{X}$ only at the locations $\mathcal{D}$
- We can seek a least-squares solution for $\boldsymbol{A}$ and $\mathbf{B}$

$$(\boldsymbol{A},\mathbf{B}) = \underset{(\mathbf{A},\mathbf{B})}{\operatorname{argmin}} \sum_{(i,j)\in\mathcal{D}} \left(X_{ij} - \sum_{k=1}^{K} A_{ik} B_{kj} \right)^2$$

- OK, but it can over-fit
- $\boldsymbol{A}$ and $\mathbf{B}$ are never unique: $\boldsymbol{AB} = (\boldsymbol{AG})(\mathbf{G}^{-1}\mathbf{B})$ for any invertible $K \times K$ matrix $\mathbf{G}$

Outline

1. Missing Data and Recommenders
2. Low-Rank Approximation
3. **Probabilistic Approach**
4. Non-Negative Matrix Factorisation



Bayesian Approach

- We can use a Bayesian approach to infer $\mathbf{A}$ and $\mathbf{B}$
- The likelihood is

$$\mathbb{P}(X_{ij}|\mathbf{A},\mathbf{B}) = \mathcal{N}\left(X_{ij}\middle|\sum_{k=1}^{K} A_{ik}B_{kj},\sigma^2\right)$$

$$\mathcal{N}(x|\mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma}\mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}$$

- σ measures the natural error in the ratings

Priors

- We also need priors

$$\mathbb{P}(\mathbf{A}) = \prod_{i,k} \mathcal{N}(A_{ik}|0,\sigma_A^2), \qquad \mathbb{P}(\mathbf{B}) = \prod_{k,j} \mathcal{N}(B_{kj}|0,\sigma_B^2)$$

- σ_A^2 and σ_B^2 explain the variation in the original matrix $\mathbf{X}$

- Note that

$$\mathbb{E}\left[\sum_{k=1}^{K} A_{ik}B_{kj}\right] = 0, \qquad \mathbb{V}\mathrm{ar}\left(\sum_{k=1}^{K} A_{ik}B_{kj}\right) = K\sigma_A^2\sigma_B^2$$

  which should be equal to the variance in X_{ij}

MAP Solution

- Calculating the posterior probability for $\mathbf{A}$ and $\mathbf{B}$ is intractable
- We can, however, compute the *maximum a posteriori* (MAP) solution, as in the last lecture

$$\mathcal{E} = \sum_{(i,j)\in\mathcal{D}} \log(\mathbb{P}(X_{ij}|\mathbf{A},\mathbf{B})) + \log(\mathbb{P}(\mathbf{A})) + \log(\mathbb{P}(\mathbf{B})) + C$$

$$= -\frac{1}{2\sigma^2}\sum_{(i,j)\in\mathcal{D}}\left(X_{ij} - \sum_{k=1}^{K} A_{ik}B_{kj}\right)^2 - \frac{1}{2\sigma_A^2}\sum_{i,k} A_{ik}^2$$

$$- \frac{1}{2\sigma_B^2}\sum_{k,j} B_{kj}^2 + C'$$

Iterative Learning

- We can maximise the log-posterior by first maximising the log-posterior with respect to $\mathbf{A}$ keeping $\mathbf{B}$ fixed

$$\frac{\partial \mathcal{E}}{\partial A_{ik}} = \frac{1}{\sigma^2} \sum_{j|(i,j)\in\mathcal{D}} \left(X_{ij} - \sum_{l=1}^{K} A_{il} B_{lj} \right) B_{kj} - \frac{1}{\sigma_A^2} A_{ik} = 0$$

- This is equivalent to

$$\sum_{l=1}^{K} \left(\frac{\sigma^2}{\sigma_A^2} \delta_{k,l} + \sum_{j|(i,j)\in\mathcal{D}} B_{lj} B_{kj} \right) A_{il} = \sum_{j|(i,j)\in\mathcal{D}} B_{kj} X_{ij}$$

Solving for $\boldsymbol{A}$

- We got

$$\sum_{l=1}^{K}\left(\frac{\sigma^2}{\sigma_A^2}\delta_{k,l}+\sum_{j|(i,j)\in\mathcal{D}}B_{lj}B_{kj}\right)A_{il}=\sum_{j|(i,j)\in\mathcal{D}}B_{kj}X_{ij}$$

- Which has the form $\mathbf{M}^i\boldsymbol{a}^i=\boldsymbol{c}^i$

$$M^i_{kl}=\frac{\sigma^2}{\sigma_A^2}\delta_{k,l}+\sum_{j|(i,j)\in\mathcal{D}}B_{lj}B_{kj},\qquad a^i_l=A_{il},\qquad c^i_k=\sum_{j|(i,j)\in\mathcal{D}}B_{kj}X_{ij}$$

- Solution is $\boldsymbol{a}^i=(\mathbf{M}^i)^{-1}\boldsymbol{c}^i$ (one $K\times K$ solve for each row i)

Each Step is Ridge Regression

- Write $\boldsymbol{b}_j$ for column j of $\mathbf{B}$ and $\boldsymbol{a}_i$ for row i of $\boldsymbol{A}$ (as a column vector)
- With $\mathbf{B}$ fixed, row i of $\boldsymbol{A}$ minimises

$$\sum_{j|(i,j)\in\mathcal{D}} \left(X_{ij} - \boldsymbol{a}_i^\mathsf{T}\boldsymbol{b}_j\right)^2 + \eta\|\boldsymbol{a}_i\|^2, \qquad \eta = \frac{\sigma^2}{\sigma_A^2}$$

- This is ridge regression: features $\boldsymbol{b}_j$, targets X_{ij}, regularisation strength η
- Its solution $\boldsymbol{a}_i = \left(\eta\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\right)^{-1}\sum_j \boldsymbol{b}_j X_{ij}$ is exactly $(\mathbf{M}^i)^{-1}\boldsymbol{c}^i$
- Alternating between $\boldsymbol{A}$ and $\mathbf{B}$ is called **alternating least squares** (ALS): each step is convex, even though the whole problem is not

Converging to a Solution

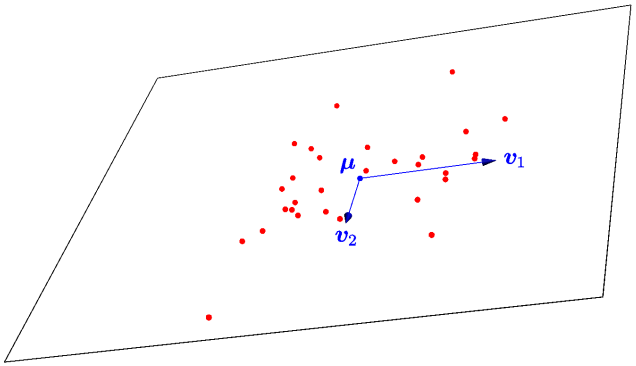
- Then solve for $\mathbf{B}$ (almost identical to solving for $\boldsymbol{A}$)
- Iterate between $\boldsymbol{A}$ and $\mathbf{B}$ until they converge
- No step can decrease the log-posterior, so it converges, usually rapidly (to a local optimum: the full problem is not convex)
- Used to predict an X_{ij} where $(i,j) \notin \mathcal{D}$
- Matrix factorisation was at the heart of the Netflix Prize winners and is still a strong baseline for recommenders

Embeddings

- Row i of $\boldsymbol{A}$ is a K-dimensional vector describing user i; column j of $\mathbf{B}$ describes item j
- The predicted rating is their inner product, $\boldsymbol{a}_i^\top \boldsymbol{b}_j$
- Items with similar vectors are rated alike: we have learnt an **embedding** of users and items
- Modern recommenders learn embeddings with neural networks, mostly from **implicit feedback** (clicks, views, purchases): a missing entry is then weak evidence of not liking, not simply unknown
- Matrix completion is also used to fill in missing survey, sensor and medical data

Outline

1. Missing Data and Recommenders
2. Low-Rank Approximation
3. Probabilistic Approach
4. **Non-Negative Matrix Factorisation**



Matrix Factorisation

- Although we have treated matrix factorisation as a matrix completion task we can also think of it as a data modelling task
- Consider a set of books in mathematics, physics and biology
- We can build a matrix of books (columns) versus words (rows), with elements being the word count
- Can use the SVD, taking $\mathbf{A}$ as the first three columns of $\mathbf{US}$ and $\mathbf{B}$ as the first three rows of $\mathbf{V}^\mathsf{T}$

SVD

	Euclid	Gauss	Feynman	Newton	Darwin	Mendel
theorem	51	101	5	10	3	1
proof	51	103	2	8	0	0
sum	22	51	60	72	5	7
vector	0	70	90	50	3	3
wave	0	1	51	92	2	0
species	0	4	5	1	161	89
fitness	0	1	3	1	35	2

$\approx$

	1	2	3
theorem	9.0	-2.2	-6.6
proof	8.8	-2.5	-7.0
sum	10.2	-1.7	3.4
vector	11.2	-2.3	3.4
wave	6.5	-1.3	7.7
species	5.2	17.7	-0.5
fitness	1.0	3.0	0.0

	Euclid	Gauss	Feynman	Newton	Darwin	Mendel
1	2.5	6.9	4.5	4.5	2.2	1.3
2	-0.8	-1.9	-0.9	-1.1	8.5	4.5
3	-3.5	-5.6	4.9	5.7	-0.3	-0.1

- Best rank-3 approximation, but the factors mix signs: hard to interpret

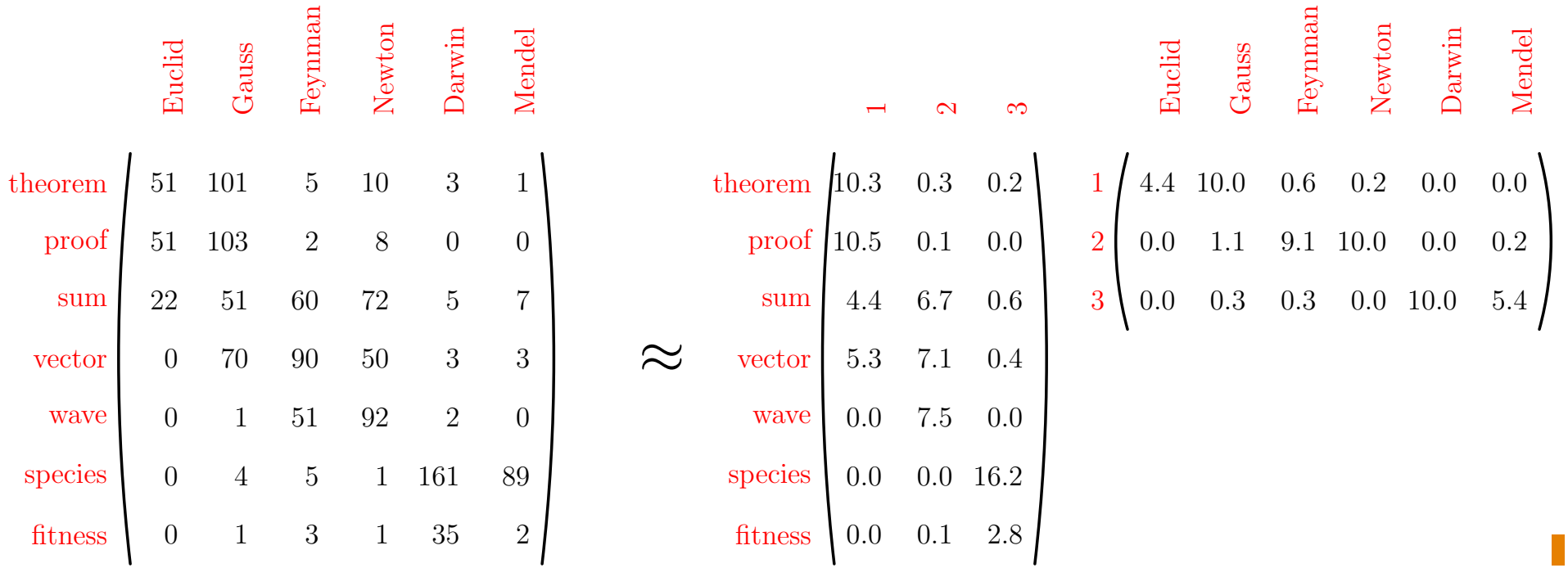
Non-Negative Matrix Factorisation

- When $\mathbf{X}$ is non-negative we can get a sparse model by minimising $E(\mathbf{A},\mathbf{B}) = \|\mathbf{X} - \mathbf{A}\mathbf{B}\|_F^2$ with all elements of $\mathbf{A}$ and $\mathbf{B}$ non-negative
- Projected gradient descent: take a gradient step, then set negative elements to zero
- Or Lee and Seung's **multiplicative updates** ($\oslash$ is element-by-element division)

$$\mathbf{B} \leftarrow \mathbf{B} \odot (\mathbf{A}^\mathsf{T}\mathbf{X}) \oslash (\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{B}), \quad \mathbf{A} \leftarrow \mathbf{A} \odot (\mathbf{X}\mathbf{B}^\mathsf{T}) \oslash (\mathbf{A}\mathbf{B}\mathbf{B}^\mathsf{T})$$

which never make an element negative

Non-Negative Matrix Factorisation



- The three factors pick out mathematics, physics and biology

Non-Negative Matrix Factorisation: Caveats

- NMF is very popular because its factors are often easier to interpret than those of the SVD
- But
  - ⋆ It cannot fit better than the SVD: by Eckart–Young the truncated SVD is the best rank-K fit (here relative errors 0.164 against 0.158)
  - ⋆ The problem is not convex: the answer depends on the starting point and on K
  - ⋆ The factors are not unique, and a "parts-based" reading of them is not guaranteed
- Always compare against simple baselines (e.g. the SVD or PCA)

Conclusion

- Structured data typically lies close to a low-dimensional subspace, and we can exploit this
- Matrix factorisation has many applications including recommender systems, data completion, and data modelling
- The truncated SVD gives the best low-rank approximation; iterating it fills in missing entries
- A Bayesian (MAP) approach helps with over-fitting, and each step of the resulting ALS is ridge regression
- Non-negative matrix factorisation gives more interpretable, though not necessarily better, factors